

DS 2500: Intermediate Programming with Data

Fall 2025

Homework 3: Data Analysis with Machine Learning

Assigned: November 4, 2025

Deadline: November 25, 2025 at 11:59 PM ET

Total Points: 1,000 pts

Overview and Learning Objectives

This assignment aims to provide hands-on experience with fundamental data science techniques using the [Abalone](#) dataset. You will explore the data, apply preprocessing techniques, build predictive models, and visualize your findings.

The age of [abalone](#) is traditionally determined by cutting the shell and counting rings under a microscope which is a tedious process. Your goal is to develop predictive models using easier-to-obtain physical measurements.

This assignment is intended to review the following concepts we covered in class:

- Similarity and distance measures.
- Scaling data.
- Correlation.
- Linear regression.
- KNN classification.
- Data visualization.

Submission Instructions

- Submit your solution as **.py** file(s) to DS2500 Gradescope by **11:59 PM ET November 25, 2025**.
- Late submission is allowed until **December 9, 2025** with a 100 point penalty. Submissions will not be accepted after the late submission deadline.
- You may submit to Gradescope as many times as you want to check your score and improve your score.

Additional Instructions

- Make sure to download both the data files (csv) from **Canvas**. You are NOT given a starter code file for this assignment but are expected to create your own Python file with the name **HW_03.py**
 - You are free to use Google Colab for this assignment. However, you will **need to download your code as .py file for submission** and not as **.ipynb**

- You **CAN** use **Pandas**, **NumPy**, **scikit-learn**, and/or **statistics** for this assignment.
- Use **print()** to test and understand your code. The answers to your questions must be printed.

Grading Policies

Homeworks are broken into three components:

1. **Accuracy (610 points)**. You'll answer quantitative questions about the dataset on Gradescope. These answers are auto-graded. Please make sure to check if your answers are correct when you submit to Gradescope. You can easily check by clicking "Save all answers" at the bottom. If your answer is correct, you will see an explanation saying that it is "correct." If there is no explanation, then you can assume it is "incorrect." You will only get full credit for the accuracy of your answers if Gradescope says "correct" for all questions.
2. **Visualization (120 points)**. You'll be asked to submit screenshots/downloads of your three Python plots. The visualization will be manually graded according to the [Python Grading Guidelines](#). Make sure that your visualization is clear, easy to follow and makes good use of labels, titles, sizes, colors, and legends if appropriate.
3. **Code Quality (270 points)**. You'll submit your code as well, which we will review and grade based on its modularity, readability, and reusability. Details are in the [DS2500 Grading Guidelines](#). *Here are some of the key things we want to see in your code:*
 - a. Using descriptive variable and function names.
 - b. Make sure to have functions to avoid repeated code and/or provide good organization. A good rule of thumb is to create a function for each question in the homework and to use helper functions to further reduce repeated code.
 - c. Call your functions in a main block or main function to actually print out the answers to the homework problems. If we were to run the code, we should see all the answers printed as output.
 - d. Make sure to include function comments as docstrings just under the function signature. Function comments should include a description of what the function does, details about arguments/parameters, and what it returns.
 - e. Add a block comment at the top of every Python file with your name, the course, the assignment, the date, and the name of the file.
 - f. Make sure to submit at least one code file (HW_03.py), but you can always choose to have more files if appropriate.

Dataset

abalone_measurements.csv contains the physical features of different abalones.

abalone_labels.csv contains the sex of abalones in the other csv file.

Data Details

Name	Data Type	Measurement	Description
abalone_id	integer		ID number of abalone
sex	nominal		M (Male), F (Female), I (Infant)
length	continuous	mm	Longest shell measurement
diameter	continuous	mm	Perpendicular to length
height	continuous	mm	With meat in shell
whole_weight	continuous	mm	Whole abalone
shucked_weight	continuous	grams	Weight of meat
viscera_weight	continuous	grams	Gut weight (after bleeding)
shell_weight	continuous	grams	After being dried
rings	integer		+1.5 gives the age in years

Python Requirements

Use Pandas for all file reading and data manipulation in this assignment. You can work in PyCharm/Visual Studio and submit a `.py` file, or work in Google Colab in a `.ipynb` file and submit a `.py` version of the file. Name the main `.py` file **HW_03.py**.

Use Pandas to Load and Preprocess the Data

Load the two Abalone dataset files into a Pandas DataFrame using `read_csv()`. You'll probably want to merge the two datasets into one dataframe using `merge()`. `abalone_id` is the column that both datasets have in common.

Filter and clean up the DataFrame: Remove rows with *any* None/empty string/NaN data, and any continuous numerical values that are 0. Note that Pandas `dropna()` only recognizes `np.nan`, `None`, and `pd.NaT` (for datetime data) as missing values.

The `rings` column is the age of the abalone. Create a new column `age` which is `rings + 1.5`. We will use `age` as our target for regression later.

Similarity & Distance Measures

For calculating the Euclidean and Manhattan distances, create functions that take in two lists of equal length and then return the distance in float. Use these functions for Problem 2 and 3.

```
def euclidean_distance(list1, list2):
    """
    Calculate Euclidean distance between two lists of equal length.

    Args:
        list1, list2: Lists of numbers with same length

    Returns:
        float: Euclidean distance
    """
    pass

def manhattan_distance(list1, list2):
    """
    Calculate Manhattan distance between two lists of equal length.

    Args:
        list1, list2: Lists of numbers with same length

    Returns:
        float: Manhattan distance
    """
    pass
```

Linear Regression & KNN Classification

For this homework, you'll build a Linear Regression model to predict the age and KNN Classifier to predict the sex of an abalone given physical measurements.

Regression

- **Feature:** `shell_weight` of abalone
- **Label:** `age` of abalone

Classifier

- **Features:** physical measurements of abalone
- **Label:** `sex` of abalone

We recommend using **scikit-learn** for this homework. It can be helpful for you to [read up on the documentation](#). Some useful functions:

- `train_test_split()`
- `LinearRegression()`, `reg.fit()`, `reg.predict()`
- `KNeighborsClassifier()`, `knn.fit()`, `knn.predict()`

- `metrics.mean_squared_error()`, `metrics.r2_score()`
- `metrics.classification_report()`, `metrics.confusion_matrix()`
- `ConfusionMatrixDisplay`

 **Note:** To make sure your results (like accuracy and F1 score) are consistent each time you run your code, please use `random_state=0` when calling `train_test_split()`. This ensures everyone has the same train/test split. Train split should be 80% of the full data while the test split should be 20%.

You can access specific columns/features from Pandas Dataframe you want to pass to train the linear regression model or classifier by doing something like the following:

```
# Select specific columns 'Name' and 'Occupation'
selected_columns_df = df[['Name', 'Occupation']]
```

Part 1: Questions about the Data

This part of your solution will be auto-graded. When you find this assignment on Gradescope, the first part will ask you the following questions; type or select your answers. You must also compute the answers to these questions programmatically and print them in your code.

Gradescope will confirm your answers are correct/incorrect. We expect everyone to get full credit on this part!

Check the output! Gradescope can be a little picky about formatting, and we don't want you to lose points for putting extra characters or whitespace in an answer. Make sure you've got the correct answer to each question for full credit.

Answer these questions in Gradescope:

1. How many rows are in the DataFrame, **after** removing any null/missing/Nan/0 values? **(30 points)**
2. After splitting the data into training (80%) and testing (20%) Pandas DataFrames `df_train` and `df_test`, how many abalones with `age=14.5` were part of the training set? Please make sure to use `random_state=0` when calling `train_test_split()`. **(50 points)**

For questions 3-5, use the training set to run the analyses and derive answers.

3. What is the Euclidean distance between abalone ID 3897 and ID 179 based on the continuous numerical features (`length`, `diameter`, `height`, `whole_weight`, `shucked_weight`, `viscera_weight`, `shell_weight`, `ring`, `age`)? What about the Manhattan distance between the same two abalones? Round to 2 decimal places. **(80 points)**
4. Using `sklearn.preprocessing`, apply `Min-Max scaling` to the numerical features. Recalculate the Euclidean and Manhattan distances between the same two abalones using the scaled data. What are the distances? Round to 2 decimal places. **(50 points)**

- Using only the training set (`df_train`) from Question 2, calculate the standard correlation matrix for the numerical features in the dataset using `corr()`. Create an [annotated heatmap](#) to visualize the correlation matrix. Which feature has the strongest positive correlation with `age` that is not `ring` or `age`? Which has the weakest positive correlation? **(100 points)**

Build your linear regression model and KNN model as described in “Linear Regression & KNN Classification”.

- What are the Mean Squared Error (MSE) and the R-squared values of your linear regression model on the testing set? Round to 2 decimal places. **(80 points)**
- For a given value of $k=2$, how many “I”s in the testing set are predicted correctly when using a KNN classifier? Please make sure to use `random_state=0` when calling `train_test_split()` with 80% for train and 20% for test. **(70 points)**
- For a given value of $k=6$, what is the f1 score for just “F”? **(80 points)**
- For possible values of $k = 3, 5, 7, 9, 11$:
 - Which value of k has the best accuracy (the % of `sex` predicted correctly)? **(70 points)**

Part 2: Visualization

Create two additional Python plots using matplotlib in addition to the heatmap from Question 4 and upload all 3 plots as screenshots/downloads.

Plot #1: Submit the annotated heatmap created in Question 4.

Plot #2: Create a scatter plot of the actual `age` vs. the predicted `age` from your linear regression for the test set, with different colors: actual (green) and predicted (orange) `age`.

Plot #3: Create a heatmap showing the [confusion matrix](#) from your KNN classifier when the value of k is optimal (out of 3, 5, 7, 9, 11) for accuracy. This optimal k should be the same as the answer for Question 9.

Part 3: Code Quality

Submit on Gradescope the code you developed to compute your answers to the Part 1 questions and to generate the plots for Part 2. Your code will be graded on modularity, readability, and reusability as discussed in class and outlined in our grading guidelines.

Tips for Success

- Start Early:** This assignment covers many fundamental concepts - give yourself plenty of time
- Test Incrementally:** Test each function as you write it
- Read Instructions Carefully:** Pay attention to which built-in functions you can and cannot use

4. **Use Helper Functions:** Break complex problems into smaller, manageable pieces if needed
5. **Comment Your Code:** Write clear comments explaining your logic
6. **30-minute Guideline:** If you get stuck on a homework problem, come by office hours or post on Piazza! We recommend you spend about 30 minutes trying to figure out a problem, and then ask for help. This will be enough time that you can try a few things to get unstuck, but not SO much time that you're banging your head against the wall. Try for 30 minutes, then ask us.